PAGE
DATE

Day 01

- DataBase life cycle

← الخطوات اللى المفروض امشى عليها ، عشان اقدر ادخل فى الاخر اعمل DB

* يعنى اى اصلا Data Base

← شويه tables مترابطه ببعض موجودة فى background بتاع APP

واى APP بيرمى البيانات بتعتمد فى DB دى ، عشان لما افتح APP تانى

الاقى البيانات موجودة من خلال انى اعمل Read ، write

* طب هل ده معناه ان كل APPs لازم يبقى ليها DB ولا فى APPs ملهاش؟

← لا عادى فى APPs ملهاش داتا بيز (Static websites)

مثال: calculator

ولكن 95% من APPs الكبيرة ليها DB

* تعالى نبدأ بقى نشوف الخطوات

1. analysis → system Analyst.

← ال client مش بيبقى فاهم هو عايز يعمل اى او بمعنى اخر هو جاى عايز

يعمل Software (APP) لمدرسة مثلا ف developers مش عايزين كده

اصلا عايزين خطوات 2 = 1+1

فبالتالى اول خطوة احنا نفهم client عايز يعمل اى ودى مسؤولية

system analysit

system analyst هو شخص عنده خبرة كبيرة فى المجال اللى هيتعمل فيه

Software (مثال هنا (APP بتاع المدرسة) فبالتالى يقدر يفهم من client

ال scope بتاع Project عبارة عن اى ويكتب الكلام ده جوه

Requirement Document

وبكده sys Analyst يبقى حل مشكلة gap بين client, developer

PAGE
DATE

2. DataBase Design → DB designer

في المرحلة دي بنحول requirement document لـ ERD

ERD → entity relationship Diagram

فيه Entity ← اي object محتاج اخزن عنه بيانات في sys بتاعي

3. DB Mapping → DB designer.

في المرحلة دي بناخد ERD ونطبق عليه شوية rules ونطلع بـ actual tables اللي هتبقى عندنا في DB

DB Mapping :

عبارة عن set of rules بطبقها على ERD فنطلع بـ actual tables / schema. اللي المفروض تبقى موجودة في DB

خلي بالك :

وانت بتعمل ERD او Mapping لو في حاجة مش واضحة او عاوز تتأكد انه اللي طلعته ده مظبوط وبيحاكي الكلام الموجود في Req Doc

Note

← ممكن نفس Req doc بطلع اكتر من ERD فبرجع لـ sys Analysit لحد ما اتأكد ان ERD السليم اللي بيحقق الـ requirements المطلوبة في sys بتاعي

← لما يبقى عندك ERD السليم ونعمل Mapping هنطلع schema واحدة لا من الاول ولا اخر هيا شوية rules وبنطبقها

* كل الخطوات دي ممكن تتعمل على ورق مش محتاجين اي tool / DB Engine

PAGE
DATE

4. DataBase implementation: → DataBase Developer

ده عايز يشغل ال DB ويبقى فعليا موجودة هنحولها من logical → Physical فهنا هنحتاجين tool

GRDBMS (Relational DataBase Management system)

مثال
- SQL server
- My SQL
- ACCess
- orcale

وعشان نشتغل على اى من tools دى لازم نتعلم SQL (لغة) structure query language (SQL)

لغة البرمجة اللى الناس بتتكلمها عشان نقدر نتعامل مع DB

- لما بنحط SQL server على الجهاز بتاعنا بيبقى اسم DB server
- المفروض الداتا بيز تبقى Centralized, Shared وللكن عشان احنا بنتعلم فكل واحد بيبقى server على جهازه

5. APPlication → APPlication programer (DB user)

GUI (interFace)

من الاول والاخر كل الحاجات دى اتعملت عشان Client وللكن هل هو ليه ان علاقة ب DB ؟

ده اكيد لا : الموظف اللى هيستخدم DB دى (end user) هو لا هيفهم tables ولا relations ولا كل الرغى ده هو عايز URL يكتبه يفتح APP (GUI) يشتغل عليه هو عمره ما هيتعامل direct مع DB

فهنا جى دور APPlication / GUI لازم يكون موجود كحلقة وصل بين Client, DB

زى مثلا الموتور بتاع العربية احنا مش بنتعامل مع الموتور على الرغم ان اهم حاجة فى العربية وللكن فى حاجات بنسوق عليها وحاجات بنحركها هيا دى اللى بنتعامل مع Motor

يبقى لازم عشان DB يكون ليها معنى ابنى عليها APP هو اللى هيكلم ال DB دى
وعلى نفس DB ممكن ابنى ١٠٠٠ Mobile / Desktop / website
او حتى الثلاثة لو عايزة
ومفيش اى تعارض بين tool اللى هتستخدمها فى بناء DB وبين اللغة
اللى هتستخدمها عشان ابنى APP

∴ Application developer هو user بتاع DB

الخلاصة :

اللى بيعمل DB ← DB developer

اللى بيستخدمها ← Application developer

بعدما APP developer يخلص app يرفعه على APP server

6. Client | End user → browser URL

ال end user بيكتب URL هو بيعمل connect على app server
ولما يحتاج يكلم DB هيبقى عبارة عن buttons وكده ال buttons
دى من الخلفية عبارة عن SQL queries هترجع تكلم DB server .

PAGE
DATE

- File Based system:

بنكتب الداتا في text files

مميزات:

- رخيص، مش محتاجين اي tool نشتري بها عشان نشتغل عليه
- مش محتاج حد عنده خبرة

بس اكيد ال file اللي هكتب فيه الداتا لازم يكون ليه format معين

→ Delimited file:

هفصل بين الداتا و بعضها ب delimiter معين

في المثال ده ←

استخدمنا comma(,)

1. ahmed, 22
2. rania, 20
3. ali, 30

→ Fixed width file:

ب اشتغل بعدد Bytes معين

يعني فلا ابقى محدد id → 2 Byte

name → 5 Byte, age → 1 Byte ...

ليه؟ ← عشان اقدر افصل بين الداتا لما اجي read / write

المشاكل؟ ←

1. difficult to search
2. low performance
3. seperated copies
4. No relationships
5. No DB Integrity
6. DB Duplication
7. long development time
8. security issues
9. bad performance
10. No constraints, rules
11. No data quality (data types)
12. Manual backup, restore
13. No standard
14. difficult integration.

PAGE
DATE

DB system:

- نبني app على DB
- هتحول من files لـ tables بينهم relationships

SID	sname	age		dID	dname
1	ahmed	20		10	SD
2	rania	22		20	HR

المميزات ؟:

1. one Standard

بقت عندي standard للداتا كلها بقت جداول مرتبطة ببعض

2. Metadata + data

الاتنين بقو موجودين جوه database

لما بنعمل ERD الموجود ده هو Metadata

data ← القيم الموجودة جوه tables ومين هعرف اوصل

للداتا من غير Metadata

3. Column → data type
4. Primary key

اي table لازم يكون ليه PK (unique, Not null)

5. Forign Key

بيضمن relationships بين tables

6. Centralized, shared DB

العيوب ؟ :

1. need expertise to used
2. DBMS itself is expensive
3. DBMS may be incompatable with any other available DBMS.

PAGE
DATE

ERD:

↳ identifies information required by business be displaying the relavent entites and relationships between them

← entity ؟

← اى object / Component يعنى احتفظ ببيانات عنه

← attributes ؟

← Property او Characteristic لل entity

→ Relationships ?

link between entities (corresponds to PK . FK equivalence in related tables)

Note

Rectangle → entity

diamond → relationship

Ellipses → attribute

يعنى اى attribute ده relationship ؟

← بيعمل لما يكون فى Property / attribute متبقى Shared بين 2 entities

زى مثلا لما نقول درجة الطالب فى كورس معين فلازم الطالب يبقى موجود والكورس كمان ، فالـ attribute ده (درجة الطالب) يتحقق وجوده فبنحط علاقة relationship بين Student, Course

Note

- واحنا بقرأ ال Req doc بيكون عندنا

nouns → entity / attribute

verbs → relationship between entities

- ينفع يبقى فى بين 2 entities اكتر من relationship بشرط تدى معانى مختلفة

types of Entities:

- Strong Entity
  - الدانا بتاعته مش هتتأثر بالتأثر ب entity تانيه
  - has a Primary key

- Weak Entity:
  - بتعتمد في وجودها على entity تانيه
  - زي مثلا dependent اللي عند Emp
  - لو الموظف مش اكيد هيتم حذف dependent اللي جيين بيه
  - do't have sufficient attributes to form a PK
  - ال Primary key بتاع weak entity بيسمى Partial key
  - Partial key: a set of attributes that can be associated with PK of an owner entity set to distingush a weak entity
  - Partial key ملوش معنى من غير Primary key

PAGE
DATE

types of attributes:

→ Simple attribute

امتى اقول على attribute انه Simple

- مش بيتجزأ
- مش بيتحسب في runtime
- مش بيتكرر لنفس الشخص

→ Composite attribute

- attribute اقدر اجزأه

name → f name, L name

- لو جمعت مكونات Composite او حطيتهم جنب بعض يديني الاصل

→ Derived attribute

- بيتحسب Calculated in run time

مثلا لو معايا birth date اقدر اجيب age

→ Multi Valued:

- حاجة بتتكرر لنفس الشخص (Phone)

→ Complex attribute

Multi Valued + Composite

Relationships:

↳ association among several entities

- May also have attributes

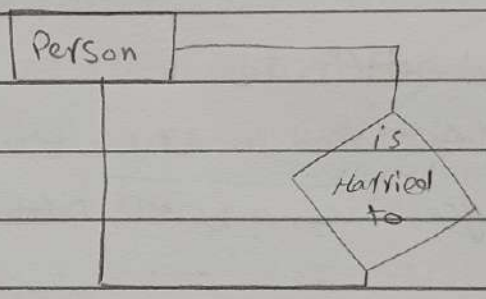
→ Degree of Relationships (الانواع)

↳ number of entities that Participate in a relationship

1. unary / recursive / self relationship

↳ between two instances on one entity type

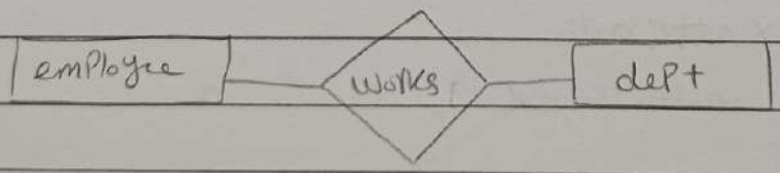
↳ A relationship in which the same entity Participate more than once



2. Binary relationships:

↳ between instances of two entity types

↳ degree of 2

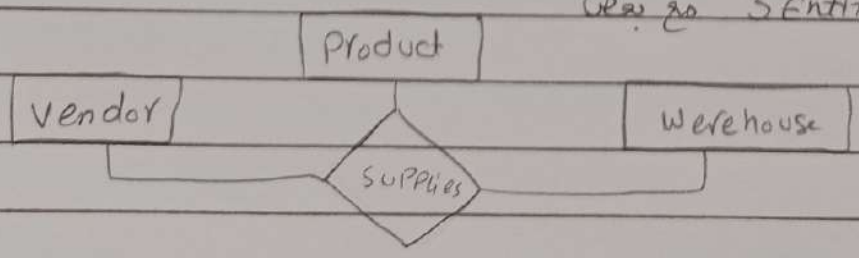


3. Ternary relationship

↳ among the instances of ~~there~~ three entity types

الاقل وجودا ، غالبا مش بتبقى موجودة الا لو في attributes

تجمع 3 Entities مع بعض



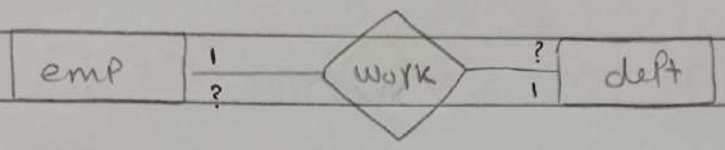
PAGE
DATE

→ Cardinality:

↳ How Many istances of one entity will or must be Connected to a single istance from the other enties

↳ مدى الترابط

• one to Many



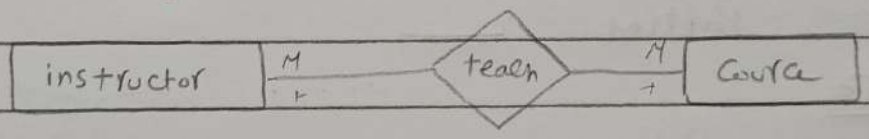
اعلى ازاى؟

- ال emp الواحد بيشتغل فى كام deft 1

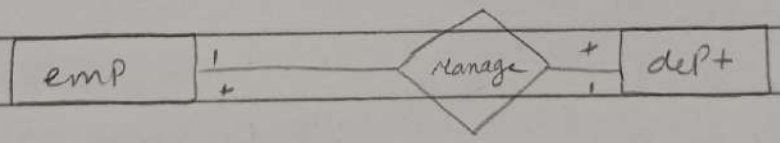
- ال deft الواحد بيشتغل فيه كام موظف M

وبعد كده اجمع

• Many to Many



• one to one



DATE

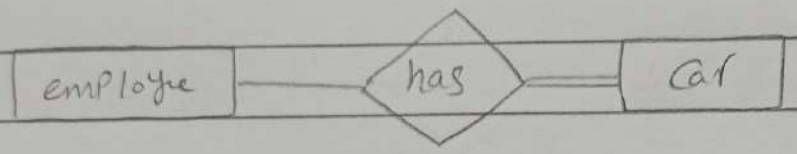
→ Participation Constraint

هل كل rows بتاع entity واحد معاها

ولا relationship

total ⟸ لو ان

Partial ⟸ لو لا



Employe may have a car → Partial

car must be assigned to particular employee → total

total ═══

Partial ———

Note

May, optional, zero or more → Partial

Must, one or more, mandatory → total.

PAGE
DATE

- Different types of Keys:

1. Candidate Key

- مرحلة قبل اختيار PK

لو عندي مثلا emp ليه name, address, id, Phone, SSN, age

بدأ اشوف مين من attributes / Columns ينفع يبقى PK

لقيت مثلا

Phone, id, SSN ⟸ يبقى اسمهم Candidate

وبعدين هختار واحد فيهم يبقى Primary

2. Composite Key

- تركيب كذا attribute مع بعض يبقو PK

- لو ملقتش اي تركيبة تحقق شروط PK هزود Column من عندي

اسمه مثلا id واطلبه هو PK

3. Foriegn Key

↳ Primary Key in another table

↳ مش لازم يكون بنفس الاسم ولكن لازم يكون بنفس data type

↳ ممكن يتكرر

↳ ممكن يكون null

↳ لازم تكون اي فيه موجودة في FK ت match فيه في PK

والعكس مش شرط يعني ممكن يكون في فيه موجودة في PK

مش موجودة في FK

4. Primary Key

↳ Column اللي بميز كل row

↳ unique, Not Null

↳ بيكون واحد بس في table

PAGE
DATE

5. Alternate key

هو Candidate key لم يتم اختياره لـ PK

يعني مثلا لو كان عندنا ال

Candidate key → Id, Email

واخترنا id بيقى هو PK

فهنقول على Email ← Alternate key

6. unique key

هي Constraint يمنع التكرار لكنه يسمح بـ null

ولكنه برضو تبقى null واحدة

7. Super key

هي اى مجموعة columns تميز كل row من table

يعني هي اى مجموعة columns لو ضمناها مع بعض وتمنع التكرار

تسمى super key

ex students

StudentID	Name	Email
1	Ali	a@mail.com
2	omar	o@Mail.Com

ال keys الممكنة

- student ID , Email , StudentID + Email , StudentID + Name
  StudentID + Name + Email

كل دول super keys لأنهم يميزوا ال row

لكن

اصغر super key بدون columns زيادة يسمى Candidate key

من المثال ده مثلا هيبقى (StudentID, Email)

Super key → Candidate key → Primary key

كل PK هو Candidate key و كل Candidate key هو super key. لكن العكس مش صح